

FRACMATH: A vectorized MATLAB framework for continuum damage mechanics with crack band regularization

Jaykumar Mavani^a, Madura Pathirage^{a,*}

^a*Gerald May Department of Civil, Construction, and Environmental Engineering, University of New Mexico, Albuquerque, NM, USA*

Abstract

FRACMATH is a MATLAB reference implementation of local isotropic continuum damage for quasibrittle solids. It combines a modified von Mises equivalent strain, exponential softening, and a direction-dependent Oliver crack-band width with vectorized triangular and tetrahedral finite elements. Its contribution is an inspectable software workflow and reproducible examples, rather than a new constitutive model. This revision exposes the sequence of assembly, factorization, equilibrium solve, damage update, and reaction calculation. A material-point test verifies the tensile/compressive strain mapping, damage irreversibility, and prescribed fracture energy over four bandwidths. A corrected two-dimensional bending run checks load-increment sensitivity and profiles the actual cost of sparse factorization separately from assembly. Three-dimensional mixed-mode and torsion examples illustrate damage localization. A corrected Abaqus UMAT run provides a code-to-code response check; job diagnostics are reported without a solver-speed claim. The revised source and scripts accompany this manuscript for inspection.

Keywords: continuum damage mechanics, crack-band regularization, finite element method, MATLAB, Abaqus UMAT, quasibrittle fracture

*Corresponding author

Email address: `mpathirage@unm.edu` (Madura Pathirage)

Required Metadata

Current code version

Nr.	Code metadata description	Value
C1	Current code version	SoftwareX development revision on GitHub main (untagged)
C2	Permanent link to code/repository used for this code version	Current corrected source: GitHub main . A permanent versioned release link and DOI must be inserted before submission. Earlier v1.0.0 is historical.
C3	Permanent link to Reproducible Capsule	Not available
C4	Legal code license	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	MATLAB; Fortran UMAT; Abaqus/Standard; Python plotting scripts
C7	Compilation requirements, operating environments, and dependencies	MATLAB R2024b used for the corrected runs; Abaqus/Standard 2024 used for the attempted UMAT rerun; Python 3 with NumPy, SciPy, and Matplotlib for revised figures
C8	If available, link to developer documentation/manual	Repository README, reproducibility guide, and theory manual in the GitHub repository
C9	Support email for questions	mpathirage@unm.edu

Table 1: Code metadata.

Current executable software version

Nr.	Executable software metadata description	Value
S1	Current executable software version	SoftwareX development revision on GitHub main (untagged)
S2	Permanent link to executable software of this version	No separate compiled executable; MATLAB source scripts are available in the GitHub repository.
S3	Legal software license	MIT License
S4	Computing platforms/operating systems	Linux, macOS, and Windows with MATLAB; Abaqus/Standard is needed only for the UMAT comparison.
S5	Installation requirements & dependencies	MATLAB R2024b used in this revision; no MATLAB toolbox beyond base MATLAB used by the 2D run.

Table 2: Executable software metadata.

Motivation and significance

FRACMATH is intended for researchers and students who need to inspect the numerical path from a strain field to a damage field and a global response. The constitutive law and crack-band principle are established methods [1, 2, 3]. The useful software question is whether the implementation makes its assumptions, update sequence, regularization length, and numerical limitations visible enough to check and modify. The package therefore includes element-level routines, a 2D bending case, two 3D examples, plotting scripts, and an Abaqus UMAT for independent comparison when its execution is verified.

Related software and scope

General-purpose finite element systems such as Abaqus [5] and OOFEM [6] provide broader element and analysis capabilities. CALFEM [8] provides an accessible MATLAB finite element teaching environment. FRACMATH offers a narrower, code-readable example of direction-dependent local crack-band damage in 2D and 3D. It is designed for teaching, replication, and testing method changes, not as a claim of faster or more accurate performance than those systems.

Software description

A scalar damage variable $\omega \in [0, 1]$ scales the elastic stress: $\boldsymbol{\sigma} = (1 - \omega)\mathbb{C}_0 : \boldsymbol{\varepsilon}$. The history variable $\kappa_{n+1} = \max(\kappa_n, \tilde{\varepsilon}_{n+1})$ makes the update irreversible. The modified von Mises measure is

$$\tilde{\varepsilon} = \frac{k-1}{2k(1-2\nu)}I_1 + \frac{1}{2k}\sqrt{\left(\frac{k-1}{1-2\nu}\right)^2 I_1^2 + \frac{12k}{(1+\nu)^2}J_2}, \quad J_2 = \frac{(e_1 - e_2)^2 + (e_2 - e_3)^2 + (e_3 - e_1)^2}{6}, \quad (1)$$

where $k = f_c/f_t$. For the 2D plane-stress element, $e_3 = -\nu(e_1 + e_2)/(1 - \nu)$. The previous 2D MATLAB and UMAT files used 1/2 instead of 1/6 in this invariant. Both revised files use Eq. (1); the 2D results below were regenerated with this correction.

The damage law is

$$\omega(\kappa) = 1 - \frac{\kappa_0}{\kappa} \exp\left[-\frac{\kappa - \kappa_0}{\varepsilon_f - \kappa_0}\right], \quad \kappa_0 = f_t/E, \quad \varepsilon_f = \frac{\kappa_0}{2} + \frac{G_F}{hf_t}. \quad (2)$$

The Oliver projected width for a three-node triangle is

$$h(\mathbf{n}) = \frac{2}{\sum_{a=1}^3 |\nabla N_a \cdot \mathbf{n}|}, \quad (3)$$

with $\mathbf{n}$ along maximum principal strain. The 3D tetrahedral implementation uses the analogous projected width. These are local regularization formulas; the material-point energy check below does not by itself prove structural mesh objectivity.

Numerical workflow and code map

The 2D driver `solver_main_3pb.m` reads the mesh and support/loading sets, precomputes elastic element matrices and shape-function gradients, and advances prescribed displacement. At each increment it (1) assembles the secant stiffness from the previous damage field, (2) factors its free block with `decomposition`, (3) solves equilibrium for the frozen damage state, (4) obtains element strains with `pagemtimes`, updates h , κ , and ω in `damage_update`, and (5) reassembles the damaged stiffness for reaction forces and the next increment. The factorization is reused only within a displacement increment. There is no outer equilibrium iteration after the damage update, so this is a sequential secant scheme rather than a fully converged Newton solve. The post-update free-DOF residual is exported for each increment.

The optional Abaqus path uses `run_3pb_abaqus_OLIVER_T3_FAST.py` to write a table of T3 shape-function gradients. The Fortran `cdm_umat_2d_OLIVER_T3_FAST.for` reads that table to compute Eq. (3); it does not use `CELENT` when the table is present. The current table initialization is restricted to one Abaqus analysis thread pending a thread-safe revision.

Material-point verification

A one-element uniaxial test checks $\tilde{\varepsilon} = \varepsilon$ in tension and $\tilde{\varepsilon} = |\varepsilon|/k$ in compression, monotone damage under unload/reload, and $h \int \sigma d\varepsilon \approx G_F$ for four prescribed projected widths. Numerical integration to a large terminal strain gives the following results for $G_F = 0.090$ N/mm.

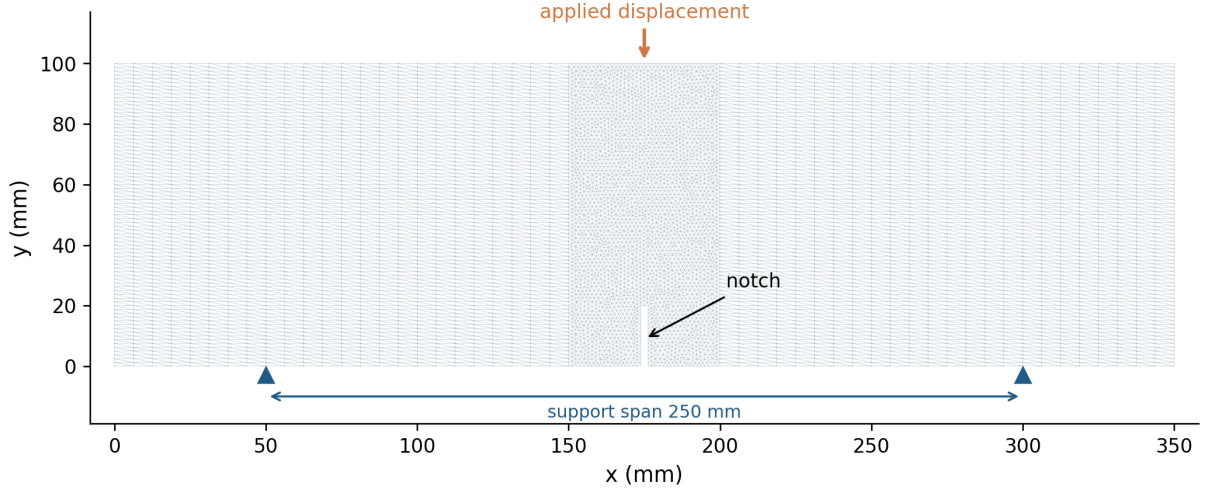
Table 3: Corrected material-point fracture energy check. The relative error is against 0.090 N/mm; this is not a structural mesh-refinement test.

h (mm)	$h \int \sigma d\varepsilon$ (N/mm)	Relative error
0.5	0.090000160	1.78×10^{-6}
1.0	0.090000160	1.77×10^{-6}
2.0	0.090000159	1.77×10^{-6}
4.0	0.090000159	1.76×10^{-6}

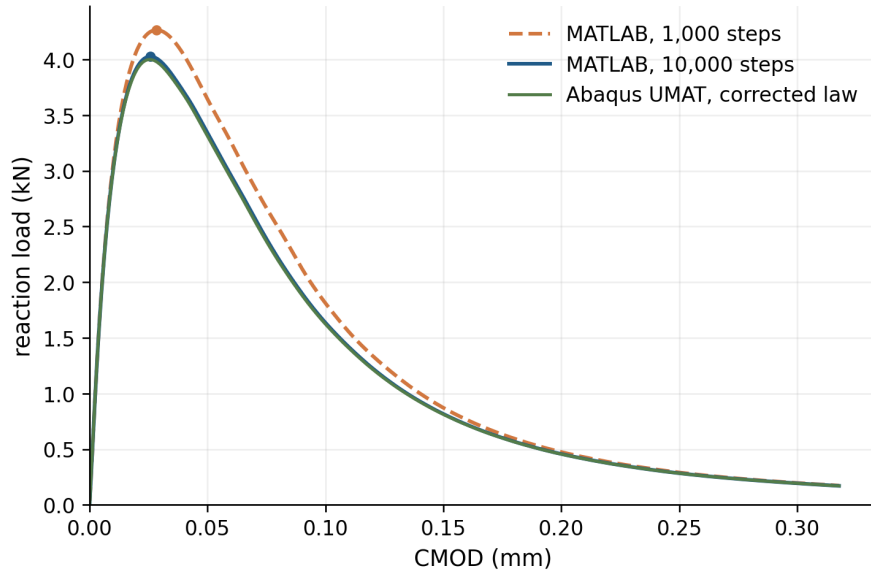
Illustrative examples

Benchmark 1: Notched 2D three-point bending

The Grégoire beam [9] has depth $D = 100$ mm, support span $S = 250$ mm, overall length 350 mm, thickness 50 mm, and a midspan notch of depth 20 mm and width 2.5 mm. Concrete parameters are $E = 37,000$ MPa, $\nu = 0.20$, $f_t = 3.5$ MPa, $f_c = 35$ MPa, and $G_F = 0.090$ N/mm. The 14,268 CPS3-element mesh has 7,319 nodes. The left support is pinned, the right is a roller, and a prescribed vertical displacement reaches -0.2 mm. CMOD is the mean horizontal displacement difference across the notch mouth. Figure 1 uses a corrected mesh drawing: the 250 mm dimension denotes support span, while 350 mm is the overall length.



(a) Geometry, mesh, supports, and imposed displacement.



(b) Corrected response and load-increment check.

Figure 1: Recomputed 2D three-point bending case.

With the corrected invariant, the 1,000-step MATLAB run peaks at 4.26 kN (CMOD 0.02835 mm, step 174), while the 10,000-step run peaks at 4.03 kN (CMOD 0.02579 mm, step 1612). This 5.9% change in peak load shows that increment size matters for the sequential scheme; these curves are numerical examples, not an increment-converged validation against experiment. The normalized post-update free-DOF residual at the peak is 0.016 for 1,000 steps and 0.002 for 10,000 steps. A corrected single-thread Abaqus/Standard 2024 UMAT run produced a peak of 4.00 kN at CMOD 0.02540 mm. The 10,000-step MATLAB peak differs by 0.68% in load and 1.51% in peak CMOD. The close response is a code-to-code check, not experimental validation.

The completed Abaqus `.msg` reports 1,136 accepted increments, 4,817 equilibrium iterations/solver passes, 4,799 matrix decompositions, one cutback, and 45 acceptances under the alternate force tolerance. Its summed per-pass sparse-solver elapsed time is 100.2 s out of 832 s reported wall clock. The remaining wall time includes element/UMAT operations, assembly, convergence and output, which the `.msg` does not separate. Thus no attribution of the remainder to the UMAT or speed ranking between codes is supported. The MATLAB run used 10,000 fixed increments, while Abaqus used automatic increments, so the two wall-clock totals are not a controlled benchmark.

Figure 2 compares damage fields at peak and at the first saved post-peak state above CMOD 0.30 mm. The Abaqus field output is taken from the nearest stored frame, while the MATLAB panels use the corrected state arrays.

The 10,000-step run used MATLAB R2024b, one computational thread, and headless output on a Windows 11 workstation with an Intel Core i9-13900 CPU and 64 GB RAM. Timing scopes around stiffness assembly, free-block factorization, damage update, and triangular solves are recorded separately in Table 4 and Figure 3. Factorization is the largest measured component at this problem size. A cross-solver speed comparison requires matched increments, convergence quality, solver settings, and timing boundaries.

Table 4: Measured MATLAB 2D timing for the corrected 10,000-step headless run. Other is total wall time minus the instrumented scopes.

Component	Time (s)	Share
Assembly	174.91	22.3%
Sparse factorization	519.54	66.1%
Damage update	17.95	2.3%
Free-block solve	71.52	9.1%
Other	1.78	0.2%
Total	785.70	100%

Benchmark 2: 3D Nooru-Mohamed Mixed Mode Test

The Nooru-Mohamed specimen is a square double edge notched concrete panel, $200 \times 200 \times 50$ mm, with two horizontal notches 25 mm long cut from opposite edges at mid height [10]; the central ligament between the notch tips experiences a mixed mode stress state. We follow load path 4c: a vertical (mode I) displacement is prescribed on the top edge and a proportional horizontal (shear) displacement on the side edges, with a shear to tension ratio $\gamma = 0.6$ and the bottom edge held fixed, ramped together under displacement control over 900 increments to a top displacement of 0.5 mm. The material is concrete with $E = 29,000$ MPa, $\nu = 0.20$, $f_t = 3.0$ MPa, $G_F = 0.110$ N/mm, and $k = f_c/f_t = 10$ (so $f_c = 30$ MPa).

The following 3D panels are retained from the AES submission as a qualitative illustration; this case was not rerun in the present revision.

Benchmark 3: 3D Notched Beam Torsion

Brokenshire’s torsion test [11, 12] uses a $400 \times 250 \times 100$ mm plain concrete beam with a diagonal notch cut through the thickness (depth 25 mm, width 5 mm) across the bottom face

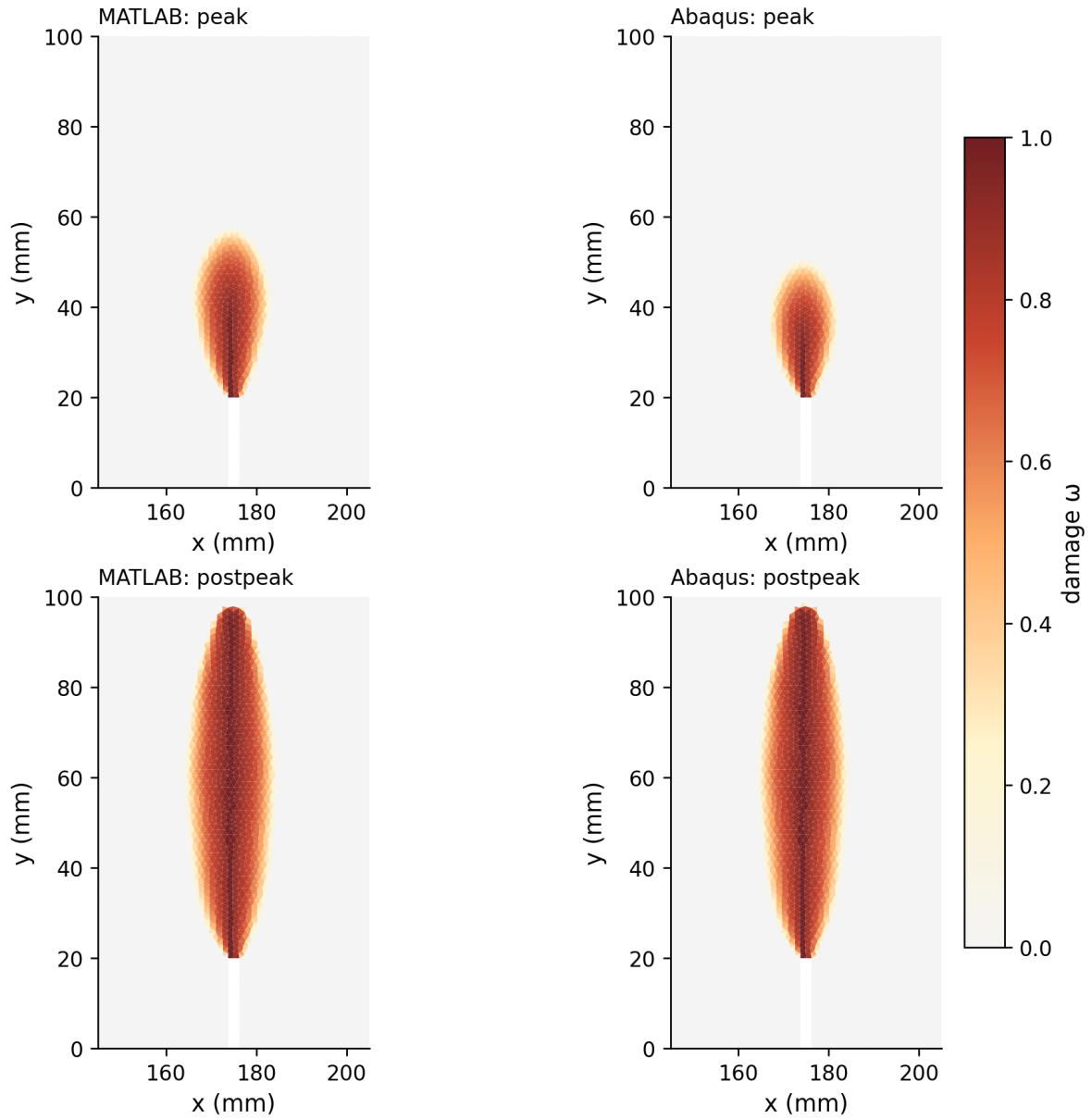


Figure 2: Corrected damage field ω in the notch region on the undeformed CPS3 mesh. Left: MATLAB; right: Abaqus/UMAT. Top: near peak load; bottom: post-peak state. Abaqus fields are the closest stored frames to the indicated response points.

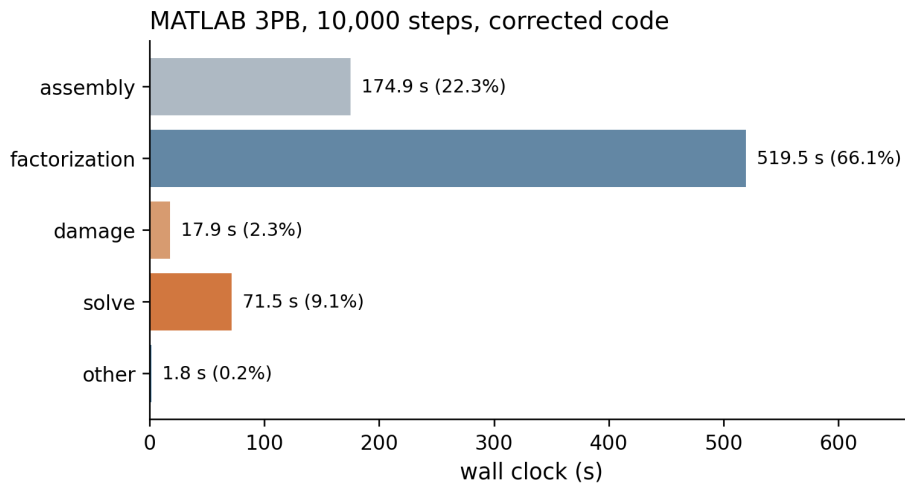
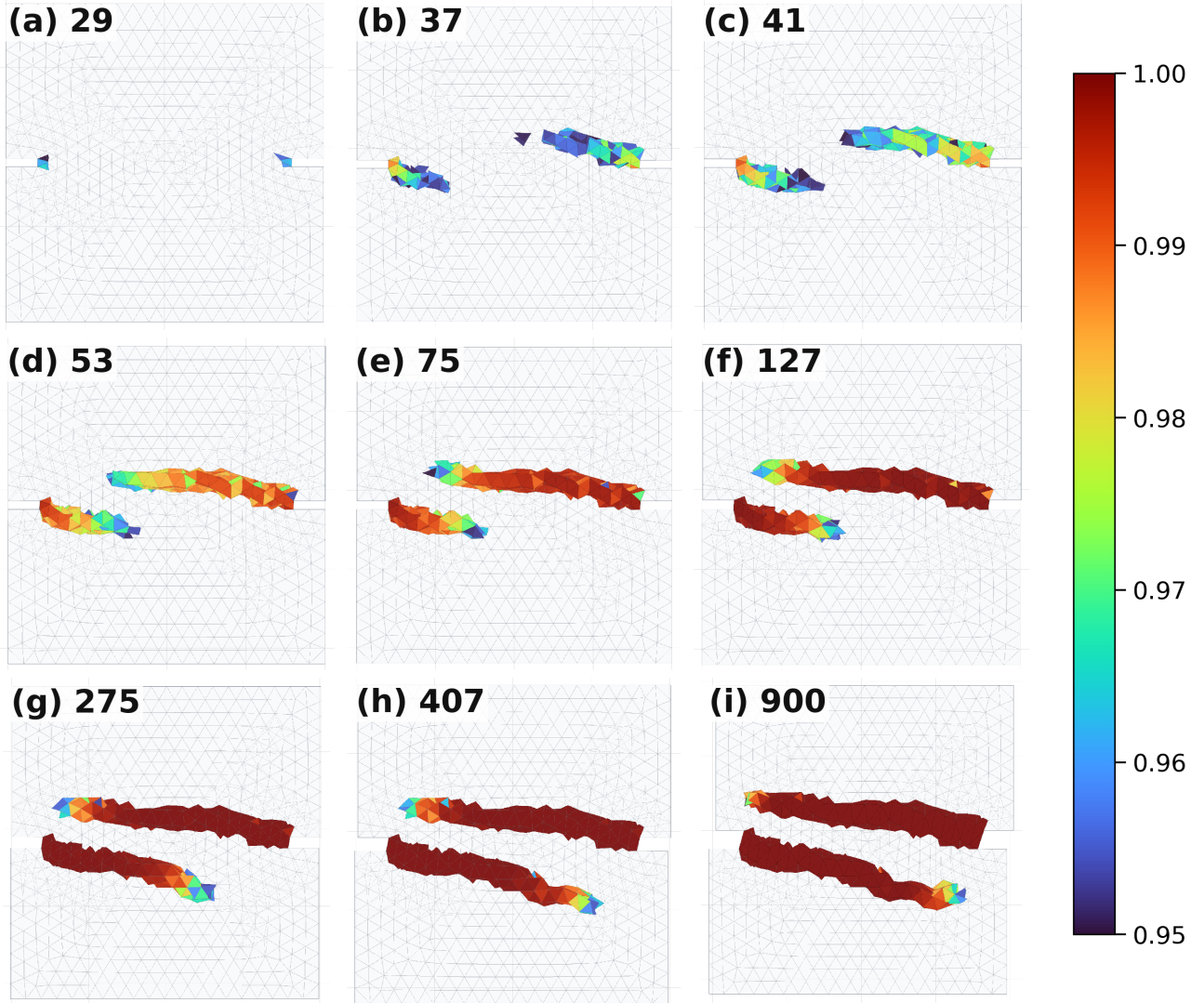
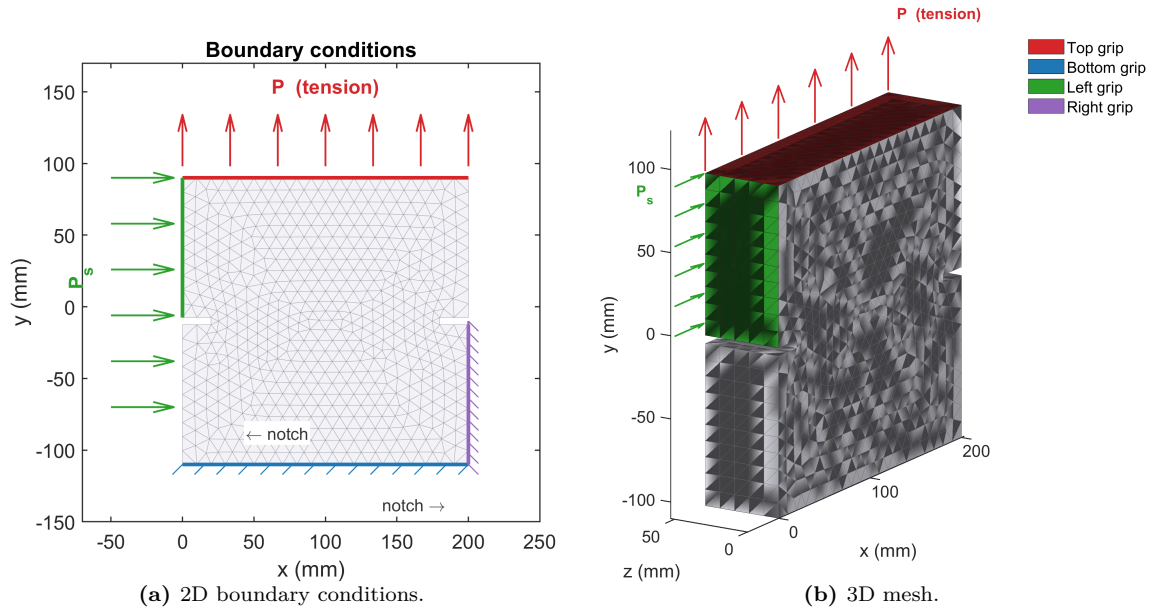
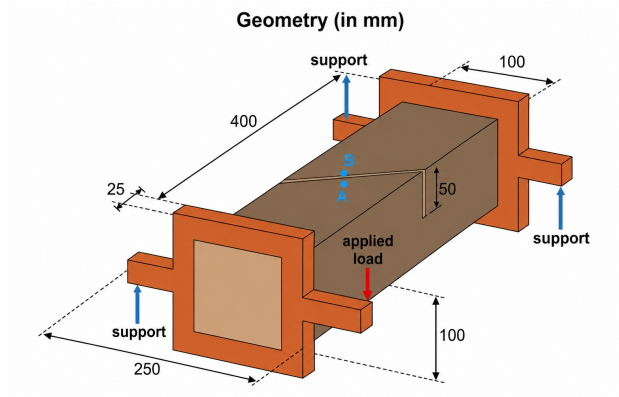


Figure 3: Measured MATLAB time by instrumented component.

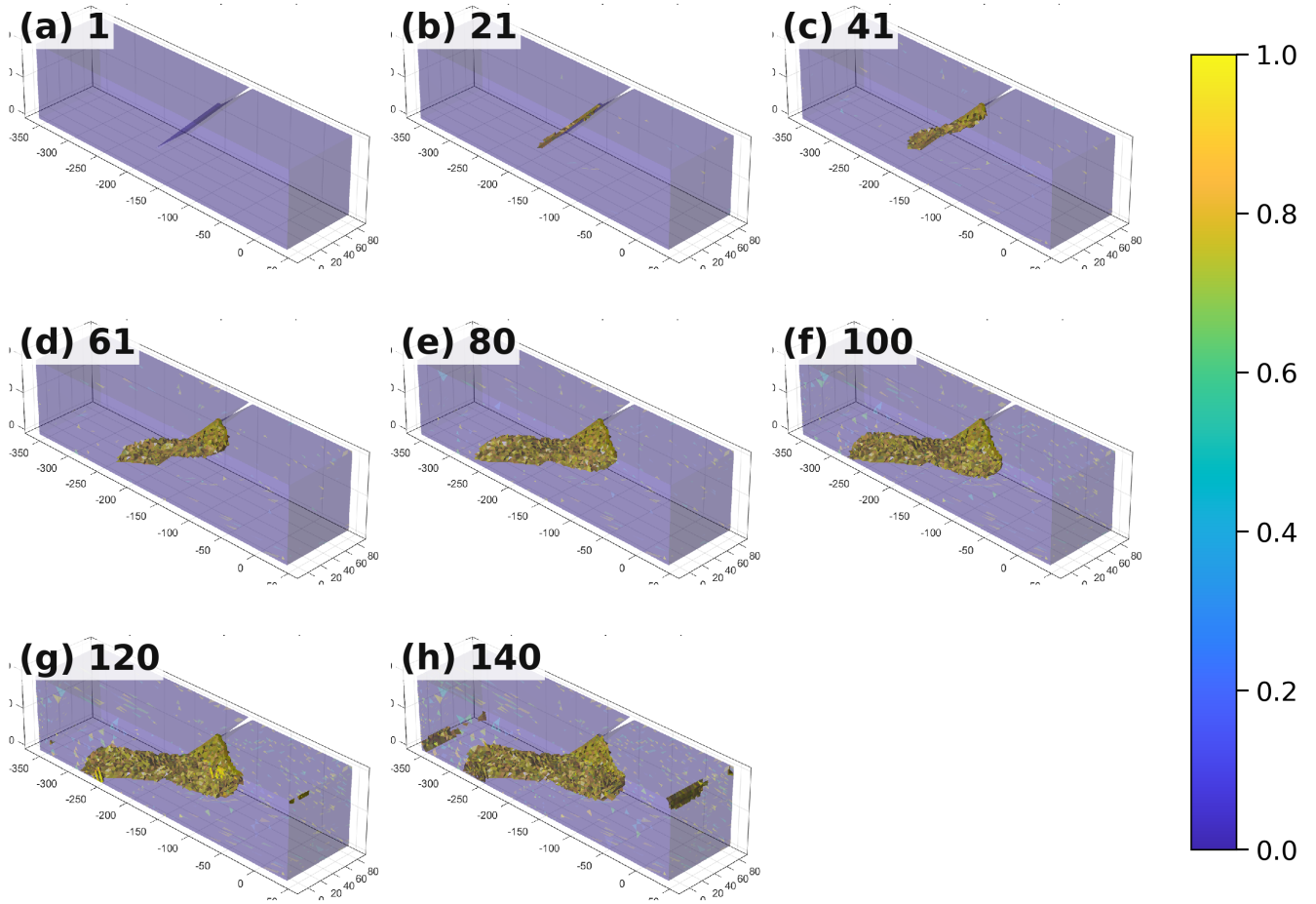


(c) Damage evolution from first localization at the notches to coalescence; panels (a)–(i), left to right and top to bottom, correspond to increments 29, 37, 41, 53, 75, 127, 275, 407, and 900.

Figure 4: Nooru-Mohamed benchmark: boundary conditions, 3D mesh, and damage evolution ω over the imposed loading history.



(a) Specimen geometry from the AES submission.



(b) Archived torsion damage sequence.

Figure 5: Brokenshire torsion example. The 3D panels are retained as qualitative figures from the AES submission; this revision did not rerun the 3D case or verify a measured torque curve.

at midspan. Three supports sit at the bottom corners and one end of the top edge, and a single point load acts on the remaining top corner (Figure 5a), so the beam is twisted and the plane of maximum principal tension turns in space around the notch front, lining up with no face of the mesh. The projected length is recomputed from the local principal direction at each integration point. This example illustrates a nonplanar damage path; its orientation and energy convergence require a mesh study.

In the model one end face is fully fixed while the opposite end face is given a prescribed rigid-body twist θ about the beam axis under rotation control, ramped to $\theta_{\text{end}} = 3.0 \times 10^{-3}$ rad over 140 increments; the reaction torque is recovered from the twisted end.

The material is plain concrete with $E = 35,000$ MPa, $\nu = 0.20$, $f_t = 3.0$ MPa, $G_F = 0.080$ N/mm, $\kappa_0 = 6.0 \times 10^{-5}$, and $k = f_c/f_t = 10$ (so $f_c = 30$ MPa), with the same exponential softening scaled element by element through the crack band length.

The TET4 mesh carries a refined layer around the notch front. The damage update is under-relaxed with a factor of 0.35 to keep the staggered fixed point iteration stable through the steep post-peak branch, and each linear solve reuses a single AMD ordering with a Cholesky factorization.

This case is a qualitative demonstration of direction-dependent crack-band regularization under nonplanar, torsion-driven crack growth: the predicted damage band nucleates at the notch front and spirals toward the loaded corner (Figure 5), qualitatively similar to the published fracture description [11]. A quantitative comparison against the measured load–crack-opening response is left for the planned nonlocal extension of the framework.

Impact

The main value of **FRACMATH** is that users can trace a complete damage calculation through mesh import, element matrices, projected bandwidth, constitutive history, sparse solve, and plotted response. The material-point test and the revised timing scopes are small, inspectable checks that expose two issues in the prior version: an incorrect 2D invariant coefficient and mislabelled factorization time. The 2D and 3D examples provide starting cases for method development and classroom use. The supplied scripts permit the figures to be regenerated from the associated data.

Limitations

The model is local, isotropic, quasistatic, and intended for monotonic loading. A single damage scalar does not represent crack closure or anisotropic degradation. The material-point energy result verifies the implemented softening calibration, but structural mesh-objectivity and 3D quantitative validation have not been demonstrated here. The 2D sequential update leaves a nonzero post-damage equilibrium residual; the 1,000/10,000-step difference shows the need for a stronger outer equilibrium iteration or an increment-convergence study before using response values as validation targets. The current Abaqus UMAT gradient-table initialization is not safe for multithreaded analysis. Performance conclusions require larger problem sizes, matched solver settings, and Abaqus diagnostic timing from completed jobs.

Conclusions

FRACMATH is an inspectable MATLAB example of local damage with an Oliver projected crack-band width. The corrected revision adds constitutive checks, a 2D increment study, and timing scopes that separate sparse factorization from assembly. The revised manuscript reports the solver’s present limitations and retains the 3D examples as qualitative demonstrations.

Data availability

The corrected 2D MATLAB source, UMAT source, mesh, material-point test, machine-readable response histories, and figure-generation script are available in the [FRACMATH GitHub repository](#). The public v1.0.0 tag is an earlier version and should not be used to reproduce the corrected 2D figures. A permanent tagged release and archive DOI must be added before SoftwareX submission.

CRedit authorship contribution statement

Jaykumar Mavani: Methodology, Software, Validation, Visualization, Writing – original draft.
Madura Pathirage: Conceptualization, Supervision, Funding acquisition.

Acknowledgements

The authors acknowledge support from the National Aeronautics and Space Administration (NASA), and the New Mexico Space Grant Consortium under grant 80NSSC22M0044.

Declaration of competing interest

The authors declare no conflict of interest.

Declaration of AI-assisted technologies

Generative AI tools were used to assist with manuscript wording, formatting, and implementation support. The scientific claims, equations, code, numerical results, figures, and references are the responsibility of the authors.

References

- [1] J. H. P. de Vree, W. A. M. Brekelmans, M. A. J. van Gils, Comparison of nonlocal approaches in continuum damage mechanics, *Computers & Structures* 55 (4) (1995) 581–588. doi:10.1016/0045-7949(94)00501-S.
- [2] J. Oliver, A consistent characteristic length for smeared cracking models, *International Journal for Numerical Methods in Engineering* 28 (2) (1989) 461–474. doi:10.1002/nme.1620280214.
- [3] Z. P. Bažant, B. H. Oh, Crack band theory for fracture of concrete, *Matériaux et Construction* 16 (3) (1983) 155–177. doi:10.1007/BF02486267.
- [4] Z. P. Bažant, J. Planas, *Fracture and Size Effect in Concrete and Other Quasibrittle Materials*, CRC Press, Boca Raton, FL, 1998.
- [5] Dassault Systèmes Simulia Corp., *Abaqus/Standard User's Manual*, Version 2024, Providence, RI, USA, 2024.
- [6] B. Patzák, Z. Bittnar, Design of object-oriented finite element code, *Advances in Engineering Software* 32 (10–11) (2001) 759–767. doi:10.1016/S0965-9978(01)00027-8.
- [7] T. A. Davis, Algorithm 832: UMFPACK v4.3—an unsymmetric-pattern multifrontal method, *ACM Transactions on Mathematical Software* 30 (2) (2004) 196–199. doi:10.1145/992200.992206.
- [8] P.-E. Austrell, O. Dahlblom, J. Lindemann, A. Olsson, K.-G. Olsson, K. Persson, H. Petersson, M. Ristinmaa, G. Sandberg, P.-A. Wernberg, *CALFEM—A Finite Element Toolbox*, Version 3.4, Lund University, Lund, Sweden, 2004.
- [9] D. Grégoire, L. B. Rojas-Solano, G. Pijaudier-Cabot, Failure and size effect for notched and unnotched concrete beams, *International Journal for Numerical and Analytical Methods in Geomechanics* 37 (10) (2013) 1434–1452. doi:10.1002/nag.2180.
- [10] M. B. Nooru-Mohamed, Mixed-mode fracture of concrete: an experimental approach, Ph.D. thesis, Delft University of Technology, 1992. URL: <https://resolver.tudelft.nl/uuid:a6a773f1-dacd-4598-aa6a-960dddf71117>.
- [11] D. R. Brokenshire, A study of torsion fracture tests, Ph.D. thesis, Cardiff University, Cardiff, United Kingdom, 1996.

- [12] A. D. Jefferson, B. I. G. Barr, T. Bennett, S. C. Hee, Three dimensional finite element simulations of fracture tests using the Craft concrete model, *Computers and Concrete* 1 (3) (2004) 261–284. doi:10.12989/cac.2004.1.3.261.